

Distributed KV Cache on Production Hardware: Mooncake Store and LMCache Results

A self-contained results brief. It reports what my own tests of distributed KV caching measured on production-grade GPUs (A100 and GB200), for two cache-management layers on vLLM: Mooncake Store through the native `KVConnector` , and LMCache with Mooncake Store as its remote tier. Every number here is self-measured.

What was tested, and how to read the numbers

The test is a controlled cold-versus-pooled comparison. Instance A serves the whole trace first against an empty pool, so it pays full prefill on every prompt; this cold-A time is the native-recompute baseline. Instance B then serves the same prompts from the now-populated pool, having never processed them locally, so any speedup must come from a cross-instance cache fetch. A and B use the same trace, the same GPU model, and the same transport within a row, so each verdict is an internal comparison on one hardware axis. The traces use distinct, long system prompts, so no instance reuses anything locally and all reuse is genuinely cross-instance. The primary metric is time to first token (TTFT) at p50, reported next to the cross-instance hit rate.

A few conditions were measured in more than one run, because the tables below were produced for different comparisons: bare Mooncake appears in both the transport comparison and the LMCache head-to-head, and the GB200 short-prefix point appears in both the transport table and the length sweep. Where the same condition recurs, the figures come from independent runs and differ only by run-to-run variance of about a millisecond; they are not meant to be identical.

Environments.

Label	Hardware	Transport	Software
A100 single node	8x A100-SXM4-80GB, NVLink, driver 570.86.15	RDMA over InfiniBand, and TCP for contrast	vLLM 0.22.0, Mooncake 0.3.11.post1
A100 cross node	2 nodes, 1x A100 each, 200 Gb InfiniBand (169 Gb/s measured)	RDMA over InfiniBand (<code>m1x5_0</code>)	vLLM 0.22.0, Mooncake 0.3.11.post1
GB200 single node	4x GB200, 186 GB each, driver 580, CUDA 13, Grace (aarch64)	RDMA over 200 Gb RoCE (<code>m1x5_2</code> ; the box's IB ports are unavailable)	vLLM 0.22.0, Mooncake 0.3.11.post1

Models are Qwen2.5-3B-Instruct unless a row says 32B, which is Qwen2.5-32B-Instruct. LMCache rows add LMCache 0.4.5.

Mooncake Store: cross-instance reuse

Reuse is correct everywhere. Across every condition in the table below the cross-instance hit rate is 98.3% and no transfer failed (the TCP failure on larger prefixes, noted after the table, is a separate and more demanding run). The rate is 98.3% rather than 100%, even with identical prompts, because vLLM caches whole 16-token blocks and always recomputes at least the final block, which cannot be

served from cache since generation needs the last position. That is a fixed loss of roughly one block per request, so it shrinks as prefixes lengthen and the hit rate climbs to 99.9% at 8,000 tokens in the sweep below. The mechanism works; the open question is whether fetching cached KV is cheaper than recomputing it, which is purely a transport and hardware question.

Condition	B hit rate	KV load (avg)	B pooled TTFT p50	A cold TTFT p50	Outcome
A100, single node, TCP	98.3%	~3,358 ms	1,843 ms	38.9 ms	net loss, ~47x slower
A100, single node, RDMA (IB)	98.3%	~2.9 ms	19.5 ms	26.9 ms	net win
A100, cross node, RDMA (IB)	98.3%	~2.4 ms	19.3 ms	26.0 ms	net win
GB200, single node, RDMA (RoCE)	98.3%	~1.6 ms	18.4 ms	15.6 ms	p50 break-even; mean 21 ms pooled vs 42 ms cold

The reuse rate is identical across rows; only the transport and the GPU generation change the verdict. Over TCP the KV load averaged about 3.3 seconds for tens of megabytes, far slower than recomputing a roughly 520-token prefix, so the pooled path is a net loss. Over RDMA with GPUDirect the same load drops to a few milliseconds, so the fetch beats recompute and B's TTFT falls below A's. The win holds across two physical machines on the InfiniBand fabric, which is the production-shaped scenario; the cross-node fabric measured 169 Gb/s and is not the bottleneck.

Two notes on reading the table. The KV-load column is a mean while the TTFT columns are medians, so on the TCP row the mean load (about 3.3 s) exceeds the median pooled TTFT (1.8 s): a heavy tail of slow transfers lifts the mean above the median request. And cold-A varies by row because instance A also writes its KV into the pool as it serves, so its TTFT is not strictly transport-independent; over TCP the slow write path is the likely reason its cold figure (38.9 ms) exceeds the RDMA rows (about 27 ms). Over RDMA the write is cheap (the KV load itself is about 2.9 ms), so cold-A there is close to pure recompute and the write overhead is not a material cost on the recommended transport; the visible inflation is a TCP artifact, and TCP is not recommended.

TCP has a hard ceiling beyond small prefixes. A separate run with longer prefixes (about 3,300 tokens, roughly 120 MB of KV each) exhausted ephemeral TCP ports on the transfer path, transfers failed, the hit rate fell to about 15%, and tail TTFT rose to 14 to 17 seconds. TCP is adequate to prove correctness on small prefixes and is not a viable transport for a real deployment. RDMA is not optional.

GB200: why I ran the more extensive tests

The A100 win is real but small (a 3B model, a 520-token prefix), and it left two questions the A100 could not answer. First, does the recommendation survive on current-generation hardware, where much faster prefill attacks the cache's advantage from the recompute side? Second, how do my numbers relate to the vendor's published GB200 results? A GB200 node was added to answer both, and because Blackwell prefill is so fast that the crossover point is no longer obvious, I measured it directly instead of asserting it.

Where the pool starts to pay, by prefix length (GB200, 3B). Fresh pool and instances per point.

Shared-prefix length	B hit rate	A cold p50	B pooled p50	Verdict at p50
~520 tokens	98.3%	15.3	18.3	recompute slightly ahead
~2,000 tokens	99.5%	43.9	24.6	pool ~1.8x faster
~4,000 tokens	99.7%	48.8	36.9	pool ~1.3x faster
~8,000 tokens	99.9%	59.8	56.5	pool marginally ahead

The crossover sits between roughly 500 and 2,000 shared tokens on this hardware: below it recompute wins, above it the pool wins. For agentic workloads with multi-thousand-token system prompts and histories, the pool pays at the median even on Blackwell. The relative win peaks at mid-length prefixes and narrows again by 8,000 tokens, because cold prefill grows sublinearly while the pooled path moves linearly more KV per request, so on this fast GPU and this fabric the two costs converge again at long prefixes.

Model size, measured (GB200, Qwen2.5-32B-Instruct). The claim that the win grows with model size was the last axis still asserted rather than measured, so the 2K and 8K points were repeated at 32B, a production-plausible serving size.

Shared-prefix length	Model	A cold p50	B pooled p50	Verdict at p50
~2,000 tokens	3B	43.9	24.6	pool ~1.8x
~2,000 tokens	32B	91.8	44.4	pool ~2.1x
~8,000 tokens	3B	59.8	56.5	pool marginal
~8,000 tokens	32B	348.3	104.9	pool ~3.3x

The narrowing seen at 3B and 8K reverses completely at 32B: prefill compute grows faster with model size than KV volume does, so the pooled fetch wins by the largest margin measured in this project, about 243 ms saved per request at the median (99.9% reuse, mean 361 versus 108 ms). This is the regime a distributed pool is actually deployed for, a real model size with long shared prefixes, and it is where the mechanism pays most decisively.

LMCache over Mooncake versus bare Mooncake Store

This compares LMCache as the cache-management layer, with Mooncake Store as its L2 remote tier, against the bare native Mooncake connector, on the same trace and transport (RDMA), so the comparison isolates what the LMCache layer adds and costs. These runs were on the A100 nodes, single node and cross node, over RDMA on InfiniBand; the GB200 prefix-length and model-size sweeps above were not repeated with LMCache. Software was vLLM 0.22.0, LMCache 0.4.5, and Mooncake 0.3.11.post1.

RDMA, same trace	B hit rate	B TTFT p50	B TTFT mean	A cold p50
Bare Mooncake, A100 single node	98.3%	19.5 ms	54 ms	26.9 ms
Bare Mooncake, A100 cross node	98.3%	20.3 ms	55 ms	26.1 ms
LMCache over Mooncake L2, A100 single node	53.4%	25.2 ms	363 ms	28.1 ms
LMCache over Mooncake L2, A100 cross node	blocked	—	—	—

Three findings, each with a mechanical cause:

- **The hit-rate gap is granularity, not correctness.** LMCache reuses full 256-token chunks and does not store partial chunks, so a roughly 500-token prompt yields exactly one full-chunk hit, about 53%. Bare Mooncake keys 16-token vLLM blocks and captures about 98% of the same prefix. The gap shrinks on longer prompts and widens on shorter ones.
- **Median is competitive, the tail is not.** LMCache's pooled p50 (25.2 ms) is close to cold recompute, but its mean (363 ms) shows a heavy retrieval tail that bare Mooncake does not have (54 ms mean). At this scale the LMCache layer costs more than its partial reuse saves.
- **The cross-node cell is blocked by an LMCache defect.** On the node remote from the master, LMCache's Mooncake connector fails to create its store client, then proceeds anyway and segfaults on first use. This was isolated to LMCache: vLLM's native connector and a standalone Mooncake client both work from the same node and image, and the failure reproduced on LMCache 0.4.5 and 0.5.0.

On raw cross-instance reuse over RDMA, bare Mooncake wins on this stack: higher reuse, a flatter latency tail, and a cleaner operational story. The capabilities that motivate LMCache (an L1 tier that survives an L2 outage, Kubernetes-native deployment through the Production Stack, and reuse of any repeated span rather than only shared prefixes) were not exercised here and remain its case for retrieval-heavy or enterprise-operability requirements.

Bottom line

Cross-instance reuse through Mooncake Store is correct (98.3% reuse at short prefixes, rising to 99.9% at multi-thousand-token prefixes) and, over RDMA with GPUDirect, faster than recomputing. It does not pay over TCP, which is a correctness transport only and fails outright on large prefixes. On current-generation Blackwell the pool pays at the median above roughly 500 to 2,000 shared tokens, and the margin grows with model size, reaching about 3.3x at 32B with 8,000-token prefixes, the regime a distributed pool is deployed for. Bare Mooncake Store is the stronger primary choice on this stack; LMCache's extra layer cost more than its partial reuse saved here, and its distinctive capabilities remain to be evaluated on their own ground.